Mathematical Formulation of the Joint Bit-Width and Fan-In Sensitivity Knapsack for FHE-Aware Neural Network Acceleration

Murat Toprak
Department of Computer Engineering, Istanbul Technical University (ITU)
toprakm23@itu.edu.tr
Accompanying Technical Formulation for PACT '26 Submission

October 19, 2026

Abstract

Fully Homomorphic Encryption (FHE) enables secure inference on encrypted neural networks, but evaluations of non-linear activations depend on computationally expensive Programmable Bootstrapping (PBS) kernels. The latency of each PBS operation scales exponentially ($2^{b_{\text{acc}}}$) with the integer accumulator bit-width b_{acc}, which is jointly governed by quantization precision and input fan-in. This document provides a complete, self-contained mathematical exposition of the joint sensitivity and constrained optimization framework laid out in `paper.tex`. We formulate the objective of minimizing overall accuracy distortion under strict cryptographic PBS budgets, derive the second-order Hessian sensitivity models for quantization and structured neuron pruning, specify the calibrated FHE PBS cost model, and demonstrate how this constrained optimization reduces to a dynamic programming/multiple-choice knapsack problem solvable in under a few milliseconds.

Contents

1 The Core Optimization Goal & High-Level Knapsack Intuition **2**
1.1 Primary Objective Statement . . . 2
1.2 Why It Solves Like a Knapsack in Milliseconds . . . 2

2 Cryptographic PBS Cost Calculation **3**
2.1 The Role of Programmable Bootstrapping (PBS) in FHE . . . 3
2.2 Exponential Scaling with Accumulator Bit-Width (b_{acc}) . . . 3
2.3 Accumulator Bit-Width Formulation: Theoretical and Calibrated . . . 4
2.4 Total Layer PBS Cost Calculation . . . 4
2.5 Structured vs. Unstructured Pruning in FHE . . . 4

3 Second-Order Hessian Sensitivity & Distortion Model **5**
3.1 Taylor Expansion of Task Loss . . . 5
3.2 Bit-Width Quantization Distortion (ΔD_{quant}) . . . 5
3.3 Structured Neuron Pruning Distortion (ΔD_{prune}) . . . 6

4 The Constrained Optimization Problem **6**
4.1 Primal Formulation: Minimal Distortion Under Budget B . . . 6
4.2 Dual Formulation: Minimal PBS Cost Under SLA Distortion ϵ_{SLA} . . . 6

5 The Knapsack Solver Algorithm & Complexity **7**
5.1 Dynamic Programming State Recurrence 7
5.2 Complexity Analysis and Runtime Validation 7

6 Empirical Validation: 192-Neuron FCNN Network **7**
6.1 Curvature Profile from paper.tex 7
6.2 Pareto Optimization Solutions Across FHE Budget Regimes 8

7 Reference Table of Mathematical Symbols **9**

---

1 The Core Optimization Goal & High-Level Knapsack Intuition

1.1 Primary Objective Statement

Let an L-layer feed-forward neural network be parameterized by weight tensors $\mathbf{W} = \{W_1, \ldots, W_L\}$ and biases $\{b_1, \ldots, b_L\}$. Our primary design goal is to minimize the total network task accuracy distortion D_{total} induced by compression, subject to the constraint that the total cryptographic Programmable Bootstrapping (PBS) execution cost does not exceed an allocated FHE execution budget B:

$$\min_{\{\mathbf{b},\mathbf{s}\}} \sum_{l=1}^{L} D_l(b_l, s_l) \quad \text{subject to} \quad \sum_{l=1}^{L} C_l(b_l, s_l, s_{l-1}) \leq B, \tag{1}$$

where:

- $b_l \in \mathcal{B} = \{3, 4, 5, 6, 8\}$ denotes the discrete bit-width precision allocated to layer l (weights and activations).
- $s_l \in \mathcal{S} = \{0.0, 0.25, 0.50, 0.75\}$ denotes the structured output neuron pruning ratio for layer l.
- $D_l(b_l, s_l)$ represents the second-order sensitivity-weighted accuracy distortion introduced by compressing layer l.
- $C_l(b_l, s_l, s_{l-1})$ represents the cryptographic FHE operational PBS cost of layer l, which is explicitly coupled to the upstream layer's sparsity s_{l-1} through incoming fan-in.
- B denotes the user-specified FHE execution budget (e.g., maximum allowable bootstrapping operations or latency threshold).
- Boundary constraints fix input dimensionality ($s_0 = 0.0$) and preserve classification logits ($s_L = 0.0$).

1.2 Why It Solves Like a Knapsack in Milliseconds

In standard Deep Learning hyperparameter optimization, searching over mixed precision and per-layer sparsity often requires hundreds of GPU hours using reinforcement learning (e.g., HAQ) or continuous gradient relaxation (e.g., DNAS). In contrast, our formulation reduces to a **Multiple-Choice Knapsack Problem (MCKP)** with first-order Markovian stage transitions:

1. **Decoupled Sensitivity Evaluation:** Rather than retraining the network to measure accuracy drop for every candidate allocation, we precompute a static second-order Hessian sensitivity profile (using Hutchinson trace and Optimal Brain Damage). Once computed, evaluating any layer distortion $D_l(b_l, s_l)$ requires only $\mathcal{O}(1)$ algebraic operations.

2. **Compact Discrete State-Space:** The candidate set for each hidden layer contains only $|\mathcal{B}| \times |\mathcal{S}| = 5 \times 4 = 20$ discrete states. For a canonical 3-layer network ($L = 3$), the entire feasible configuration space comprises:

$$|\Omega| = 20 \times 20 \times 5 = 2{,}000 \quad \text{candidate configurations.} \tag{2}$$

3. **Sub-Millisecond Evaluation:** Because the cost function $C_l(b_l, s_l, s_{l-1})$ and distortion function $D_l(b_l, s_l)$ are closed-form vectorizable expressions, evaluating all 2,000 points and identifying the Pareto-optimal minimum requires only elementary CPU arithmetic. In our benchmark implementation (`qat/allocation/allocator.py`), this entire knapsack grid search executes in 1.2 **to** 1.8 **milliseconds** on an ordinary single-core x86 CPU.

4. **Polynomial Scaling for Deep Networks:** For arbitrary network depth L, the 1D Dynamic Programming recurrence (detailed in Section 5) evaluates in $\mathcal{O}(L \cdot |\mathcal{B}| \cdot |\mathcal{S}|^2 \cdot \frac{B}{\Delta c})$ pseudo-polynomial time, running in < 15 ms for $L = 20$.

2 Cryptographic PBS Cost Calculation

To understand the cost function $C_l(b_l, s_l, s_{l-1})$, we examine how neural networks execute under Torus Fully Homomorphic Encryption (TFHE) compilers, specifically Zama Concrete ML.

2.1 The Role of Programmable Bootstrapping (PBS) in FHE

In an L-layer neural network, each layer computes:

$$z_l = W_l \, a_{l-1} + b_l, \tag{3}$$

$$a_l = \sigma(z_l), \tag{4}$$

where $\sigma(\cdot)$ is a non-linear activation function (such as ReLU).

- **Affine Operations (Leveled LWE):** The matrix-vector multiplication in Eq. (3) is evaluated homomorphically via plaintext-ciphertext multiplications and homomorphic additions over Learning With Errors (LWE) ciphertexts. These operations are computationally inexpensive and require no bootstrapping.
- **Non-Linear Activations (PBS):** FHE ciphertext additions and multiplications only support linear arithmetic. Non-linear activation functions $\sigma(\cdot)$ require *Programmable Bootstrapping* (PBS). PBS homomorphically evaluates a univariate Look-Up Table (LUT) over the torus $\mathbb{T} = \mathbb{R}/\mathbb{Z}$ via Blind Rotation and Key Switching while simultaneously resetting the accumulated ciphertext noise.

PBS accounts for $> 95\%$ of the total cryptographic computation during encrypted inference.

2.2 Exponential Scaling with Accumulator Bit-Width ($b_{\mathbf{acc}}$)

To prevent decryption errors during bootstrapping, the ciphertext must accommodate the full integer range of the linear accumulator without modulus overflow:

$$z_{l,i} = \sum_{j=1}^{k_l} W_{l,i,j} \, a_{l-1,j} + b_{l,i}. \tag{5}$$

In TFHE, the cryptographic parameters (LWE dimension n, GLWE dimension k, polynomial degree N, and ciphertext modulus q) must be chosen to ensure 128-bit quantum security while

supporting the accumulator bit-width b_{acc}. The operational cost of the bootstrapping kernel (which performs FFT-based polynomial multiplications in $\mathbb{Z}_q[X]/(X^N+1)$) scales exponentially with b_{acc}:

$$\text{Cost}_{\text{unit}}(b_{\text{acc}}) \propto 2^{b_{\text{acc}}}. \tag{6}$$

Every additional bit of accumulator depth doubles the operational runtime of each bootstrapping lookup.

2.3 Accumulator Bit-Width Formulation: Theoretical and Calibrated

Theoretical Dynamic Range: When weights and activations at layer l are quantized to b_l bits, the product $W_{l,i,j}a_{l-1,j}$ occupies up to $2b_l$ bits before scaling. Summing over active incoming fan-in k_l requires:

$$b_{\text{acc}}^{\text{worst}}(l) \approx b_l + \lceil \log_2(k_l) \rceil + \text{margin}, \tag{7}$$

where k_l is the number of active input connections feeding each neuron of layer l.

Incoming Fan-In Coupling (k_l): Crucially, the active fan-in into layer l is directly dictated by the structured sparsity s_{l-1} of the previous layer:

$$k_l = \text{fan_in}_l(s_{l-1}) = \max\left(1,\ \lceil N_{\text{in},l} \cdot (1 - s_{l-1}) \rceil\right), \tag{8}$$

where $N_{\text{in},l}$ is the unpruned input dimension.

Calibrated Statistical Model: In practice, inputs and weights are standardized (zero-mean). By the Central Limit Theorem, the accumulation variance scales with $\sqrt{k_l}$ rather than k_l. Calibrating against 42 compiled single-layer circuits in Concrete ML via least-squares regression yields the predictor ($R^2 > 0.98$):

$$b_{\text{acc}}(l, b_l, s_{l-1}) = \min\left(16,\ \max\left(b_l + 2,\ b_l + \lceil \log_2(\text{fan_in}_l(s_{l-1})) \rceil - 3\right)\right), \tag{9}$$

where the -3 offset captures the empirical dynamic range reduction achieved by standardized distribution calibration.

2.4 Total Layer PBS Cost Calculation

A PBS operation is executed once for each active output neuron in layer l that undergoes non-linear activation. Under structured output sparsity s_l, the number of active neurons is:

$$N_{\text{PBS}}(l, s_l) = \lceil N_{\text{out},l} \cdot (1 - s_l) \rceil. \tag{10}$$

Multiplying the number of active output neurons by the unit PBS cost yields the operational cost for layer l:

$$\boxed{C_l(b_l, s_l, s_{l-1}) = \lceil N_{\text{out},l} \cdot (1 - s_l) \rceil \cdot 2^{b_{\text{acc}}(l, b_l, s_{l-1})}} \tag{11}$$

The total network PBS cost is the sum over all L layers:

$$C_{\text{total}}(\mathbf{b}, \mathbf{s}) = \sum_{l=1}^{L} C_l(b_l, s_l, s_{l-1}), \quad \text{with } s_0 = 0. \tag{12}$$

2.5 Structured vs. Unstructured Pruning in FHE

As summarized in Table 1, unstructured pruning is incapable of reducing cryptographic PBS invocations because neurons retain at least one non-zero input. In contrast, structured pruning simultaneously drops PBS invocations in layer l and lowers the accumulator bit-width b_{acc} in layer $l+1$.

Table 1: Architectural comparison of pruning paradigms in FHE.

Metric	Unstructured Pruning (Prior Work)	Structured Neuron Pruning (Ours)
Weight Sparsity	High (individual weights zeroed)	Block-sparse (entire rows removed)
Active Neurons (N_{PBS})	$N_{\text{out},l}$ (unchanged, 100% active)	$\lceil N_{\text{out},l}(1-s_l)\rceil$ (**physically reduced**)
Active Fan-in (k_l)	$N_{\text{in},l}$ (virtually all neurons connected)	$\lceil N_{\text{in},l}(1-s_{l-1})\rceil$ (**physically reduced**)
Accumulator Depth b_{acc}	Unchanged	Reduced by 1–2 bits
Total PBS Invocations	384 PBS lookups	96 **PBS lookups (4× fewer)**
Unit PBS Cost	Fixed at high baseline	Reduced exponentially by $2\times$–$4\times$

3 Second-Order Hessian Sensitivity & Distortion Model

To compute distortion without expensive retraining during search, we formulate a second-order sensitivity model based on the Hessian matrix of the task loss $\mathcal{L}$.

3.1 Taylor Expansion of Task Loss

Let $\mathbf{W}^*$ be the converged network weights. Perturbations $\Delta\mathbf{W}$ caused by quantization and pruning alter the task loss according to:

$$\Delta\mathcal{L} = \mathcal{L}(\mathbf{W}^* + \Delta\mathbf{W}) - \mathcal{L}(\mathbf{W}^*) = \nabla_{\mathbf{W}}\mathcal{L}(\mathbf{W}^*)^T\Delta\mathbf{W} + \frac{1}{2}\Delta\mathbf{W}^T H\,\Delta\mathbf{W} + \mathcal{O}(\|\Delta\mathbf{W}\|^3), \quad (13)$$

where $H = \nabla^2_{\mathbf{W}}\mathcal{L}(\mathbf{W}^*)$. Since the model is trained to convergence, $\nabla_{\mathbf{W}}\mathcal{L}(\mathbf{W}^*) \approx 0$. Thus:

$$\Delta\mathcal{L} \approx \frac{1}{2}\Delta\mathbf{W}^T H\,\Delta\mathbf{W}. \quad (14)$$

Assuming block-diagonal separability across layers:

$$D_{\text{total}}(\mathbf{b},\mathbf{s}) = \sum_{l=1}^{L} D_l(b_l,s_l) = \sum_{l=1}^{L}\left(\Delta D_{\text{quant}}(l,b_l) + \Delta D_{\text{prune}}(l,s_l)\right). \quad (15)$$

3.2 Bit-Width Quantization Distortion (ΔD_{quant})

Quantizing layer l to b_l bits introduces quantization noise ΔW_l with variance proportional to 2^{-2b_l}. Taking the expectation over isotropic perturbations:

$$\mathbb{E}\left[\frac{1}{2}\Delta W_l^T H_l \Delta W_l\right] \propto \text{Tr}(H_l)\cdot 2^{-2b_l}. \quad (16)$$

Hutchinson Trace Estimator: The full Hessian H_l is too large to materialize. We compute its trace using the Hutchinson stochastic estimator over $N = 20$ Rademacher random vectors $v^{(s)} \in \{-1,+1\}^{\dim(W_l)}$:

$$\text{Tr}(H_l) \approx \frac{1}{N}\sum_{s=1}^{N} v^{(s)^T} H_l\, v^{(s)}, \quad (17)$$

where Hessian-vector products $H_l v^{(s)}$ are evaluated via Pearlmutter's autograd double-backward graph (HVP $= \nabla_{W_l}(\nabla_{W_l}\mathcal{L}\cdot v^{(s)})$).

Setting maximum precision $b_{\max} = 8$ bits as the zero-distortion baseline:

$$\boxed{\Delta D_{\text{quant}}(l,b_l) = \text{Tr}(H_l)\cdot\left(2^{-2b_l} - 2^{-2b_{\max}}\right)} \quad (18)$$

3.3 Structured Neuron Pruning Distortion (ΔD_{prune})

Using Optimal Brain Damage (OBD) with diagonal Hessian elements:

$$H_{w_{i,j},w_{i,j}} \approx \frac{1}{N}\sum_{s=1}^{N}\left(H_l v^{(s)}\right)_{i,j} \odot v_{i,j}^{(s)}, \tag{19}$$

the saliency of individual weight $w_{i,j}$ is:

$$\mathcal{I}(w_{i,j}) = \frac{1}{2} H_{w_{i,j},w_{i,j}}\, w_{i,j}^2. \tag{20}$$

The saliency of output neuron i in layer l is the sum of its incoming weight importances:

$$\mathcal{I}(\text{neuron } i) = \sum_{j=1}^{N_{\text{in},l}} \mathcal{I}(w_{i,j}). \tag{21}$$

Pruning the $k_l = \lfloor N_{\text{out},l} \cdot s_l \rfloor$ least important neurons (denoted by index set $\mathcal{P}_l(s_l)$) induces the pruning distortion:

$$\boxed{\Delta D_{\text{prune}}(l, s_l) = \sum_{i \in \mathcal{P}_l(s_l)} \mathcal{I}(\text{neuron } i)} \tag{22}$$

The total layer distortion is the additive sum:

$$\boxed{D_l(b_l, s_l) = \Delta D_{\text{quant}}(l, b_l) + \Delta D_{\text{prune}}(l, s_l)} \tag{23}$$

4 The Constrained Optimization Problem

4.1 Primal Formulation: Minimal Distortion Under Budget B

The primal optimization problem seeks the joint bit-width vector $\mathbf{b}^*$ and structured sparsity vector $\mathbf{s}^*$ that minimize total network accuracy distortion while strictly respecting the FHE PBS budget B:

$$\begin{aligned}
\min_{\mathbf{b},\mathbf{s}} \quad & \sum_{l=1}^{L}\left[\text{Tr}(H_l)\left(2^{-2b_l} - 2^{-2b_{\max}}\right) + \sum_{i \in \mathcal{P}_l(s_l)} \mathcal{I}(\text{neuron}_{l,i})\right] \\
\text{subject to} \quad & \sum_{l=1}^{L} \lceil N_{\text{out},l}(1-s_l)\rceil \cdot 2^{b_{\text{acc}}(l,b_l,s_{l-1})} \le B, \\
& b_l \in \mathcal{B} = \{3,4,5,6,8\}, \quad \forall l \in \{1,\dots,L\}, \\
& s_l \in \mathcal{S} = \{0.0, 0.25, 0.50, 0.75\}, \quad \forall l \in \{1,\dots,L-1\}, \\
& s_0 = 0.0, \quad s_L = 0.0.
\end{aligned} \tag{24}$$

4.2 Dual Formulation: Minimal PBS Cost Under SLA Distortion ϵ_{SLA}

In operational deployment, a system architect often specifies a strict SLA accuracy bound (e.g., maximum allowable loss degradation ϵ_{SLA}). The dual problem minimizes FHE execution cost subject to this accuracy bound:

$$\begin{aligned}
\min_{\mathbf{b},\mathbf{s}} \quad & \sum_{l=1}^{L} \lceil N_{\text{out},l}(1-s_l)\rceil \cdot 2^{b_{\text{acc}}(l,b_l,s_{l-1})} \\
\text{subject to} \quad & \sum_{l=1}^{L} D_l(b_l, s_l) \le \epsilon_{\text{SLA}}, \\
& b_l \in \mathcal{B}, \quad s_l \in \mathcal{S}, \quad s_0 = 0.0, \quad s_L = 0.0.
\end{aligned} \tag{25}$$

5 The Knapsack Solver Algorithm & Complexity

5.1 Dynamic Programming State Recurrence

Because the cost of layer l depends only on (b_l, s_l) and the upstream sparsity s_{l-1}, the problem defines a first-order Markov Decision Process.

Let $V(l, s_{l-1}, c)$ denote the minimum accumulated distortion from layer l through layer L, given upstream sparsity s_{l-1} and remaining budget capacity $c \in [0, B]$:

$$V(l, s_{l-1}, c) = \min_{\substack{(b_l, s_l) \in \mathcal{B} \times \mathcal{S} \\ C_l(b_l, s_l, s_{l-1}) \le c}} \left\{ D_l(b_l, s_l) + V\Big(l+1,\ s_l,\ c - C_l(b_l, s_l, s_{l-1})\Big) \right\}, \tag{26}$$

with boundary base conditions:

$$V(L+1, s_L, c) = \begin{cases} 0, & \text{if } c \ge 0, \\ +\infty, & \text{if } c < 0. \end{cases} \tag{27}$$

5.2 Complexity Analysis and Runtime Validation

- **Precomputation Complexity:** Precomputing the distortion table requires sorting neuron importance scores once per layer ($\mathcal{O}(L \cdot N_{\text{out}} \log N_{\text{out}})$). For the 192-neuron network, this takes < 0.1 ms.
- **Search Complexity:** For $L = 3$, Algorithm 1 iterates through $|\mathcal{B}|^3 \cdot |\mathcal{S}|^2 = 5^3 \cdot 4^2 = 2{,}000$ points. Each iteration consists of three additions, two comparisons, and table indexing. The entire loop executes in **1.2 to 1.8 ms**.
- **General Depth Complexity:** For arbitrary L, solving the dynamic program with budget step Δc requires $\mathcal{O}(L \cdot |\mathcal{B}| \cdot |\mathcal{S}|^2 \cdot \frac{B}{\Delta c})$ transitions. For $L = 20$ and $B/\Delta c = 1{,}000$, this finishes in < 15 ms.

6 Empirical Validation: 192-Neuron FCNN Network

6.1 Curvature Profile from paper.tex

Evaluating the 3-layer architecture (`fc1`: $784 \to 192$, `fc2`: $192 \to 192$, `fc3`: $192 \to 10$) yields the empirical curvature profile in Table 2:

Table 2: Layer-wise Hessian curvature breakdown ($N = 20$ Hutchinson samples).

Layer	Parameters	Trace $\text{Tr}(H_l)$	% Curvature	Normalized $\overline{H}_l$	Density Rank
`fc1`	150,528	**29.78**	**56.6%**	0.00020	#3
`fc2`	36,864	13.87	26.4%	0.00038	#2
`fc3`	1,920	8.98	17.0%	**0.00468**	**#1 (23.7×)**

Curvature Implications:

1. `fc1` possesses 56.6% of the curvature trace. Aggressive quantization on `fc1` corrupts input features irreparably.
2. `fc3` has a 23.7× higher normalized trace density ($\overline{H}_l = \text{Tr}(H_l)/\text{params}$) than `fc1`. Output logits are ultra-sensitive to rounding noise and must not be pruned ($s_3 = 0$).

Algorithm 1 Fast Knapsack Grid Search for 3-Layer Architectures

Require: Hessian traces $\{\mathrm{Tr}(H_l)\}$, neuron OBD scores $\{\mathcal{I}(\mathrm{neuron}_{l,i})\}$, budget B, candidate sets $\mathcal{B}, \mathcal{S}$.

Ensure: Optimal configuration $(\mathbf{b}^*, \mathbf{s}^*)$, minimal distortion D^*, and total cost C^*.

Step 1: Precompute Distortion Look-Up Tables (LUTs):
for $l \in \{1, 2, 3\}$ **do**
  for $(b, s) \in \mathcal{B} \times \mathcal{S}$ **do**
    Precompute $D_l(b, s) \leftarrow \Delta D_{\text{quant}}(l, b) + \Delta D_{\text{prune}}(l, s)$.
  end for
end for
Initialize $D^* \leftarrow \infty$, $(\mathbf{b}^*, \mathbf{s}^*) \leftarrow$ None.
Step 2: Vectorized Grid Traversal ($|\Omega| = 2{,}000$ points):
for $(b_1, s_1) \in \mathcal{B} \times \mathcal{S}$ **do**
  $C_1 \leftarrow \lceil 192 \cdot (1 - s_1) \rceil \cdot 2^{b_{\text{acc}}(1, b_1, s_0=0)}$
  if $C_1 > B$ **then continue**
  end if
  for $(b_2, s_2) \in \mathcal{B} \times \mathcal{S}$ **do**
    $C_2 \leftarrow \lceil 192 \cdot (1 - s_2) \rceil \cdot 2^{b_{\text{acc}}(2, b_2, s_1)}$
    if $C_1 + C_2 > B$ **then continue**
    end if
    for $b_3 \in \mathcal{B}$ **do** ▷ Logits layer cannot be pruned: $s_3 = 0$
      $C_3 \leftarrow 10 \cdot 2^{b_{\text{acc}}(3, b_3, s_2)}$
      $C_{\text{total}} \leftarrow C_1 + C_2 + C_3$
      if $C_{\text{total}} \leq B$ **then**
        $D_{\text{total}} \leftarrow D_1(b_1, s_1) + D_2(b_2, s_2) + D_3(b_3, 0)$
        if $D_{\text{total}} < D^*$ **then**
          $D^* \leftarrow D_{\text{total}}, \quad C^* \leftarrow C_{\text{total}}$
          $(\mathbf{b}^*, \mathbf{s}^*) \leftarrow \big((b_1, b_2, b_3), (s_1, s_2, 0)\big)$
        end if
      end if
    end for
  end for
end for
return $(\mathbf{b}^*, \mathbf{s}^*), D^*, C^*$

6.2 Pareto Optimization Solutions Across FHE Budget Regimes

Applying Algorithm 1 discovers the optimal configurations in Table 3:
Key Takeaways:

- $4\times$ **Fewer PBS Calls:** By setting $s_1 = 75\%$ and $s_2 = 75\%$, active hidden neurons shrink from $192 \to 48$, reducing PBS lookups from $384 \to 96$.
- **Accumulator Bit-Width Reclaimed:** Shrinking fan-in by $4\times$ recovers 2 bits of accumulator headroom ($\lceil \log_2(k_l) \rceil$).
- **Higher Accuracy at Lower Cost:** The recovered headroom allows allocating higher precision (up to 8 bits) on curvature-dense layers, achieving +2.6% higher test accuracy while executing at $4\times$ lower computational cost.

Table 3: Knapsack allocation vs. Zama baseline (Stoian et al., 2023).

Regime	Method	(b_1, b_2, b_3)	(s_1, s_2, s_3)	Active PBS	FHE Cost	M
Low-Latency	Baseline [1]	$(2, 2, 2)$	$(0\%, 0\%, 0\%)$	384	24,576	
($b_{\text{acc}} \leq 6$b)	**Ours**	$(\mathbf{3}, \mathbf{3}, \mathbf{4})$	$(\mathbf{75\%}, \mathbf{75\%}, \mathbf{0\%})$	**96 (4× fewer)**	**6,144**	**94.8%**
Balanced	Baseline [1]	$(2, 2, 2)$	$(0\%, 0\%, 0\%)$	384	49,152	
($b_{\text{acc}} \leq 7$b)	**Ours**	$(\mathbf{4}, \mathbf{4}, \mathbf{6})$	$(\mathbf{62.5\%}, \mathbf{62.5\%}, \mathbf{0\%})$	**144 (2.7× fewer)**	**18,432**	**97.6%**
High-Accuracy	Baseline [1]	$(2, 2, 2)$	$(0\%, 0\%, 0\%)$	384	98,304	
($b_{\text{acc}} \leq 8$b)	**Ours**	$(\mathbf{4}, \mathbf{8}, \mathbf{6})$	$(\mathbf{75\%}, \mathbf{75\%}, \mathbf{0\%})$	**96 (4× fewer)**	**24,576**	**98.2%**

Table 4: Comprehensive glossary of mathematical symbols.

Symbol	Description	Domain / Definition
L	Total number of neural network layers	$L \in \mathbb{Z}^+$
W_l	Weight matrix at layer l	$\mathbb{R}^{N_{\text{out},l} \times N_{\text{in},l}}$
b_l	Quantization precision for layer l	$b_l \in \mathcal{B} = \{3, 4, 5, 6, 8\}$
s_l	Structured output neuron sparsity	$s_l \in \mathcal{S} = \{0.0, 0.25, 0.50, 0.75\}$
k_l	Active incoming fan-in into layer l	$\max(1, \lceil N_{\text{in},l}(1 - s_{l-1}) \rceil)$
$b_{\text{acc}}(l)$	Calibrated accumulator bit-width	$\min(16, \max(b_l + 2, b_l + \lceil \log_2(k_l) \rceil - 3))$
$N_{\text{PBS}}(l)$	Active output neurons (PBS invocations)	$\lceil N_{\text{out},l} \cdot (1 - s_l) \rceil$
$C_l(b_l, s_l, s_{l-1})$	FHE operational PBS cost for layer l	$N_{\text{PBS}}(l) \cdot 2^{b_{\text{acc}}(l)}$
B	Total allowable FHE PBS budget	$B \in \mathbb{R}^+$
H_l	Layer Hessian matrix	$\nabla^2_{W_l} \mathcal{L} \in \mathbb{R}^{\dim(W_l) \times \dim(W_l)}$
$\text{Tr}(H_l)$	Hutchinson stochastic trace estimate	$\frac{1}{N} \sum_{s=1}^{N} {v^{(s)}}^T H_l v^{(s)}$
$\mathcal{I}(w_{i,j})$	OBD weight importance	$\frac{1}{2} H_{w_{i,j}, w_{i,j}} w_{i,j}^2$
$\mathcal{I}(\text{neuron } i)$	Neuron importance score	$\sum_j \mathcal{I}(w_{i,j})$
$\Delta D_{\text{quant}}(l)$	Quantization distortion penalty	$\text{Tr}(H_l) \cdot (2^{-2b_l} - 2^{-2b_{\max}})$
$\Delta D_{\text{prune}}(l)$	Pruning distortion penalty	$\sum_{i \in \mathcal{P}_l(s_l)} \mathcal{I}(\text{neuron } i)$
$D_l(b_l, s_l)$	Total layer distortion	$\Delta D_{\text{quant}}(l, b_l) + \Delta D_{\text{prune}}(l, s_l)$
D_{total}	Total network accuracy distortion	$\sum_{l=1}^{L} D_l(b_l, s_l)$
C_{total}	Total network FHE execution cost	$\sum_{l=1}^{L} C_l(b_l, s_l, s_{l-1})$

7 Reference Table of Mathematical Symbols

References

[1] A. Stoian, J. Bossuat, R. Sirdey, and C. Gouert. Deep neural networks for encrypted inference with tfhe. In *Cryptology ePrint Archive, Paper 2023/257*, 2023.